BẢNG PHÂN CÔNG CHI TIẾT TUẦN 1

Tối Ưu Cục Bộ (Local Search) & Delta Evaluation

Lưu ý: Làm việc **song song**.

1. Nhánh 1: Thao Tác Cấu Trúc & Logic Toán Tử (Hà Thu)

Mục tiêu: Xử lý thuần túy mảng dữ liệu (List), rút ra, cắm vào và bắt lỗi ràng buộc cơ bản của cặp điểm.

File làm việc chính: `operators.py` **(Tạo mới)**

Các hàm cần hoàn thiện (Code Python):

- `relocate(route, node_idx, target_idx)`:
  - *Nhiệm vụ:* Xóa phần tử tại `node_idx` và chèn nó vào `target_idx` trên `route`. Trả về mảng `route` mới.
- `swap(route1, route2, node_idx1, node_idx2)`:
  - *Nhiệm vụ:* Đổi chỗ 2 phần tử giữa 2 chuyến xe khác nhau.
- **Logic Đặc Biệt (Pair Mode - Bắt buộc):**
  - Viết thêm hàm phụ `get_passenger_pair(route, node)`. Nếu `node` là điểm đón khách P_i^n, phải tìm ngay điểm trả D_i^n kề đó.
  - Khi thực hiện `relocate` hoặc `swap` đối với `passenger`, bắt buộc phải bốc cả cụm $[P_i^n, D_i^n]$ đem đi để giữ luồng đi thẳng.
- **Phân loại không gian:**
  - Viết nhóm `intra_route_operators()`: Chỉ xáo trộn vị trí trong cùng 1 xe.
  - Viết nhóm `inter_route_operators()`: Lấy điểm từ xe này chuyển sang xe khác.

2. Nhánh 2: Toán Học Tối Ưu & Lõi Hill Climbing (Dũng)

Mục tiêu: Tối ưu hóa tốc độ thuật toán để đạt số vòng lặp tối đa, tránh việc lặp lại các phép tính $O(n)$.

File làm việc 1: `evaluator.py` (Tạo mới hoặc sửa từ utils.py)

Các hàm cần hoàn thiện (Code Python):

- `calculate_delta_cost(node_A, node_B, node_C, insert_node)`:
  - *Nhiệm vụ:* Tính toán độ chênh chi phí $O(1)$. Ví dụ chèn `X` vào giữa `A` và `B`: $\Delta_{cost} = cost(A, X) + cost(X, B) - cost(A, B)$.
- `calculate_delta_time(...)`:
  - *Nhiệm vụ:* Tính độ chênh lệch thời gian đến các điểm sau điểm được chèn. Xử lý logic cộng dồn thời gian dịch vụ (Service Time).
- `check_capacity_delta(...)`:
  - *Nhiệm vụ:* Viết hàm kiểm tra tải trọng cộng dồn cục bộ, ngắt ngay lập tức (return False) nếu vượt quá tải trọng xe mà không cần duyệt hết mảng.

File làm việc 2: `hill_climbing.py` (Tạo mới)

Các hàm cần hoàn thiện (Code Python):

- `class HillClimbingSolver`:
  - *Nhiệm vụ:* Kế thừa bộ dữ liệu từ `ProblemInstance`. Nhận đầu vào là mảng `routes` từ `GreedySolver`.
  - Vòng lặp `while not reached_local_optima`:
    1. Gọi thử các hàm trong `operators.py` của Thu để sinh ra cấu hình hàng xóm.
    2. Truyền vị trí thay đổi vào `evaluator.py` để lấy điểm chênh lệch $\Delta_{Benefit}$.
    3. Nếu $\Delta_{Benefit} > 0$ và thỏa mãn mọi ràng buộc: Chấp nhận (`accept`) lộ trình mới.
    4. Đếm số bước lặp không cải thiện (`non_improving_steps`). Nếu vượt quá ngưỡng N thì dừng thuật toán.

3. Quy trình Ghép Code (Merge Phase)

1. Dũng import `operators.py` vào `hill_climbing.py`.
2. Truyền kết quả giải pháp cơ sở (`baseline_solution`) từ `GreedySolver` vào mô hình.
3. Chạy `python main.py` (khai báo `HillClimbingSolver`) trên các bộ testcase nhỏ để đảm bảo quá trình tính toán $O(1)$ trùng khớp với kết quả nếu duyệt $O(n)$ thông thường.

4. Quy ước làm việc song song (Tránh Conflict Code)

- **Giao tiếp qua Interface (Chốt Đầu vào / Đầu ra):** Hai bên chốt trước tên hàm, tham số truyền vào và kiểu dữ liệu trả về. Dũng khi viết vòng lặp trong `hill_climbing.py` cứ gọi các hàm `relocate` hoặc `swap` và mặc định xem như Hà Thu đã code đúng bên trong. Không ai phải ngồi chờ ai code xong ruột hàm.
- **Không sửa file chéo:** Hà Thu chỉ tập trung toàn bộ code xử lý mảng vào file `operators.py`. Dũng chỉ tập trung vào `evaluator.py` và `hill_climbing.py`. Tuyệt đối không tự ý thay đổi các thuộc tính ở các file cấu trúc chung nếu chưa thống nhất.
- **Test độc lập (Mock Testing):**
  - *Hà Thu:* Cứ dùng hàm kiểm tra tính hợp lệ vét cạn cũ để kiểm tra xem hàm cắt/ghép mảng của mình có đang giữ đúng cụm điểm đón/trả kề nhau hay không, chưa cần quan tâm tốc độ.
  - *Dũng:* Khi test vòng lặp Hill Climbing và hàm tính độ chênh chi phí, giả lập hàm của Thu trả về một mảng ngẫu nhiên để kiểm tra xem vòng lặp có tự động dừng đúng ngưỡng hay không.
- **Quy trình ghép code một chiều:** Đến bước ráp nối cuối ngày, Hà Thu không cần đụng vào file thuật toán. Dũng sẽ là người nhận file `operators.py` của Thu, import các hàm toán tử vào `hill_climbing.py`, truyền đầu vào từ `GreedySolver` và trực tiếp chạy debug tổng thể để bắt lỗi.